

```

char addrbuf[INET6_ADDRSTRLEN];

sin = (struct sockaddr_in *) &addr_change->spc_addr;
const char *ap = inet_ntop(AF_INET, &sin->sin_addr,
    addrbuf, INET6_ADDRSTRLEN);

printf("Peer Addr Change: %s state: %d, error: %d\n", ap,
    addr_change->spc_state, addr_change->spc_error);
}
break;

case SCTP_REMOTE_ERROR:
    remote_error = &notif->sn_remote_error;
    printf("Remote Error: err: %u len: %u\n",
        ntohs(remote_error->sre_error), ntohs(remote_error->sre_len));
    break;

case SCTP_SHUTDOWN_EVENT:
    printf("Shutdown Event.\n");
    break;

default:
    printf("Unknown event type: %u\n", notif->sn_header.sn_type);
    break;
}
}

int main()
{
    int fd, new_fd, ret, len, msg_flags;

    struct sockaddr_in own, cli_addr;
    struct sctp_sndrcvinfo sri;

    char buffer[100]; /* To read/write data */

    /* Open socket. Note: IPPROTO_SCTP use */
    if ((fd = socket(AF_INET, SOCK_STREAM, IPPROTO_SCTP)) == -1)
    {
        perror("socket");
        exit(1);
    }

    own.sin_family = AF_INET;
    own.sin_port = htons(2000); /* Some available port */
    own.sin_addr.s_addr = INADDR_ANY;
    if (bind(fd, (struct sockaddr *)&own, sizeof(own)) == -1)
    {
        perror("bind");
        exit(1);
    }

    if (listen(fd, 1) < 0)
    {
        perror("listen");
        exit(1);
    }

    /* Enable all events */
    struct sctp_event_subscribe event;
    event.sctp_data_io_event = 1;
    event.sctp_association_event = 1;
    event.sctp_address_event = 1;
    event.sctp_send_failure_event = 1;
    event.sctp_peer_error_event = 1;
    event.sctp_shutdown_event = 1;
    event.sctp_partial_delivery_event = 1;
    event.sctp_adaption_layer_event = 1;

    while(1)
    {
        new_fd = accept(fd, (struct sockaddr *)&cli_addr, (&len));
        if (new_fd > 0)
        {
            printf("New connection from: %s\n", inet_ntoa(cli_addr.sin_addr));
        }

        /* Subscribe to events */
        if (setsockopt(new_fd, IPPROTO_SCTP, SCTP_EVENTS, &event, sizeof(event)) != 0)
        {
            perror("setsockopt");
            return (-1);
        }

        /* Receive msg from client */
        memset(buffer, 0x0, sizeof(buffer));
        ret = sctp_recvmsg(new_fd, buffer, sizeof(buffer),
            (&cli_addr), &cli_addr, (&len), &sri, &msg_flags);
        if (ret < 0)
        {
            perror("sctp_recvmsg");
            continue;
        }

        /* Handle events, if any */
        if (msg_flags & MSG_NOTIFICATION)
        {
            handle_event(buffer);
            continue;
        }

        printf("Received %d bytes on stream no %d\n", buffer, sri.sinfo_stream);

        /* Echo back received data */
        len = sizeof(cli_addr);
        ret = sctp_sendmsg(new_fd, buffer, ret, (&cli_addr),
            len, 0, 0, sri.sinfo_stream, 0, 0);
        if (ret < 0)
        {
            perror("sctp_sendmsg");
        }
        else
        {
            //
        }
    }
}

```